

ATOM ORACLE BOOK · v3



# Codex Orange Book

## ฉบับ Atom สอน Axe

จากตำรา Codex สู่ workflow ที่ใช้ได้จริงใน Oracle School

3-Skill Pipeline · Write · Cover · Cheatsheet

Atom Oracle — AI Oracle ไม่ใช่มนุษย์  
สำหรับ Axe · 30 มิถุนายน 2026

# คำนำ

---

เล่มนี้คือ v3 ที่ทำใหม่หลังจากรอบก่อนยังมีขอบขาวและยังไม่ถึง final polish รอบนี้ใช้ครบสามชั้นของ Oracle book: เขียนตัวเล่มด้วย `oracle-write-complete-book` , ทำปกและ visual pass ด้วย `oracle-book-cover` , และทำสูตรโกงท้ายเล่มด้วย `oracle-cheatsheet` .

แหล่งข้อมูลมาจาก `/learn -deep` ของ repo `alchaincyf/codex-orange-book` , official Codex manual cache, session evidence จาก `$dig` , และบทเรียนจากการแก้รอบก่อนหน้า หนังสือเล่มนี้ไม่ใช่การแปล Orange Book ทั้งเล่ม แต่เป็นการกลั่นให้ Axe เอาไปใช้และสอนต่อได้จริง.

# บทที่ 1 — ทำไมต้องเขียนเล่มใหม่

หนังสือเล่มนี้เริ่มจากความผิดพลาดระดับงานเล็ก ๆ ครับ ตอนแรก Atom ทำ cheatsheet ให้ Axe เพราะตีความว่า “สอนเร็ว ๆ” ก็พอ แต่พอ Axe ถามกลับว่า “ไม่ได้ใช้ cheatsheet หรือ ? เขียนหนังสือควรทำยังไง ?” จุดสำคัญก็ชัดเจนขึ้นทันที: ถ้าโจทย์คือหนังสือ เราต้องทำงานแบบหนังสือ ไม่ใช่ทำโน้ตสั้นแล้วเรียกมันว่าหนังสือ

ความต่างอยู่ที่ความรับผิดชอบต่อผู้อ่าน cheatsheet เอาไว้ช่วยจำ แต่หนังสือต้องพาคนอ่านเดินจากไม่รู้ไปรู้ไปสุ่ทำเองได้ ต้องมีหลักฐาน ต้องมีโครงสร้าง ต้องบอกข้อจำกัด และต้องมีทางให้ตรวจสอบซ้ำได้

แหล่งตั้งต้นของเล่มนี้คือ `codex-orange-book` ซึ่งเป็น repo artifact มากกว่า repo โค้ด มันมี PDF, EPUB, HTML, README และ media ครบ แต่ไม่มี source code หรือ build pipeline ให้รัน นี่เป็นบทเรียนแรก: ไม่ใช่ทุก repo ที่ clone มาแล้วจะเป็น codebase บาง repo คือ “ผลลัพธ์ที่ publish แล้ว” วิธีเรียนจึงต้องต่างกัน

ถ้าเรียน repo โค้ด เราถามว่า entry point อยู่ไหน dependency คืออะไร test อยู่ตรงไหน แต่ถ้าเรียน repo หนังสือ เราต้องถามว่า artifact ไหนเป็น source of reading, version ไหน, ภาษามีที่ชัด, และข้อมูลส่วนไหนอาจเก่าแล้ว

เล่มนี้จึงไม่ใช่การแปล Orange Book ทั้งเล่ม แต่เป็นการกลั่นให้เป็น “ฉบับ Atom สอน Axe” คือเอาแกนของ Codex, AGENTS.md, Skills, MCP, Goal, Safety และ Memory มาอธิบายเป็น workflow ที่ใช้จริงในบ้าน Atom ได้

กฎของเล่มนี้มีสามข้อ

หนึ่ง ห้ามเขียนจากความจำล้วน ทุกข้อใหญ่ต้องโยงกลับไปยังไฟล์ที่เรียนมา official manual cache หรือ session evidence

สอง ห้ามขายฝันว่า AI ทำได้ทุกอย่าง ต้องบอก boundary เช่น credential, production, payment, secret, destructive action และ full access

สาม หนังสือที่ดีต้องช่วยให้ Axe สอนคนอื่นต่อได้ ถ้าอ่านจบแล้วยังต้องถามใหม่ทุกชั้น แปลว่าเล่มนี้ยังไม่พอ

นี่คือเหตุผลที่เราทำใหม่ ไม่ใช่เพราะ cheatsheet ไร้ค่า แต่เพราะ cheatsheet เป็นท้ายเล่ม ไม่ใช่ตัวเล่ม

## บทที่ 2 — Codex ไม่ใช่ปุ่มเดียว

สิ่งที่คนเริ่มใช้ Codex มักพลาดคือคิดว่า Codex เป็นเครื่องมือเดียว จริง ๆ แล้ว Codex เป็นชุดของพื้นผิวการทำงาน แต่ละพื้นผิวเหมาะกับงานคนละชนิด

ถ้างานอยู่ใน repo บนเครื่องเรา เช่น แก้ไฟล์ รัน test อ่าน log หรือทำ patch ขนาดเล็ก CLI/local surface คือสนามที่ตรงที่สุด จุดแข็งคือเห็น filesystem จริง ทำงานซ้ำได้ และ verify ด้วย command ได้

ถ้างานต้องดูหลาย branch หลาย agent หลาย review Codex App จะเหมือนห้องบัญชาการ มันดีสำหรับเห็นภาพรวม แต่ไม่ควรใช้แทนการตรวจหลักฐานจริง UI ทำให้เรารู้สึกว่าทุกอย่างเรียบร้อยได้ง่ายกว่าความจริง

ถ้างานยาวและฝากไปทำ asynchronous เช่นทำ PR, review, หรือ cloud task พื้นผิวแบบ cloud/web มีประโยชน์ แต่ทุกครั้งที่ออกจากเครื่อง local ต้องคิดเรื่องสิทธิ์ network, repo access, secret และ approval boundary

ถ้างานผูกกับ editor ให้ใช้ IDE extension จุดแข็งคืออยู่ใน flow ของ developer จุดอ่อนคือผู้ใช้มักลืมว่า context ใน editor ไม่เท่ากับ context ของทั้ง repo

ถ้างานต้องมองหน้าเว็บ dashboard หรือ browser flow จึงค่อยใช้ browser/computer-use surface อย่าใช้เป็นค่าเริ่มต้นเพราะมันแพงกว่า ซ้ำกว่า และตรวจซ้ำยากกว่า command line

กฎที่ Atom ใช้กับ Axe คือ “เลือก surface ที่เล็กที่สุดที่พิสูจน์งานได้” ถ้า `rg`, `sed`, `python`, `npm test` ตอบได้ ก็ไม่ต้องเปิดเครื่องมือที่ใหญ่กว่า ถ้างานต้องแตะ account ภายนอกหรือ production ก็ต้องหยุดที่เส้นอนุมัติ

การเลือก surface คือการเลือกความเสี่ยงด้วย ยิ่ง surface มีอำนาจมาก ความรับผิดชอบในการพิสูจน์ก็ยิ่งมาก คำว่าเร็วไม่ใช่เหตุผลพอที่จะเปิด full access เสมอไป

สรุปบทนี้คือ Codex ไม่ใช่ปุ่มเดียว มันคือแผงควบคุม คนใช้ที่ดีไม่ใช่คนกดทุกปุ่ม แต่คือคนรู้ว่าปุ่มไหนพอสำหรับงานนี้

## บทที่ 3 — Loop ที่ทำงานไม่หลุด

แกนของงาน Codex ที่ไม่ใช่ prompt สวย แต่คือ loop ที่ตรวจได้ สำหรับ Atom loop พื้นฐานคือ `Read → Plan → Patch → Test → Report`

Read หมายถึงอ่านของจริงก่อน อ่านไฟล์ อ่าน log อ่าน issue อ่าน docs อ่าน session evidence ไม่ใช่เริ่มจากเดาว่า project น่าจะเป็นแบบนั้น ในงาน Orange Book รอบนี้ Read คือการใช้ `/learn --deep` ให้ subagents อ่าน artifact repo หลายมุม แล้วเขียน docs ออกมาเป็นหลักฐาน

Plan คือการจัดลำดับก่อนแต่ละไฟล์ งานเล็กอาจเป็น plan ในหัวและรายงานสั้น ๆ แต่งานหนังสือต้องมี outline ก่อน draft เพราะถ้าไม่ outline ก่อน บทจะซ้ำกัน บางบทไม่มี proof และสุดท้ายหนังสือจะกลายเป็นแซทยาว

Patch คือการแก้จริงหรือสร้าง artifact จริง ถ้าเป็นโค้ดก็แก้ไฟล์ ถ้าเป็นหนังสือก็สร้าง chapter files, outline, PDF สิ่งสำคัญคือ artifact ต้องอยู่ใน path ที่ส่งต่อได้ ไม่ใช่อยู่ในคำตอบของโมเดลอย่างเดียว

Test คือการพิสูจน์ โค้ดต้อง test/build หนังสือต้อง render PDF, ตรวจ font, ตรวจขนาดไฟล์, เปิดอ่านคร่าว ๆ, และดูว่า source path มีจริง ถ้าส่ง Discord ต้องมีไฟล์จริงให้ bridge แแนบ ไม่ใช่ local path อย่างเดียว

Report คือการสรุปผลพร้อมหลักฐาน บอกไฟล์ที่สร้าง คำสั่งที่รัน ผลตรวจ และข้อจำกัด รายงานที่ดีไม่ต้องยาว แต่ต้องทำให้คนอ่านเชื่อได้โดยไม่ต้องเดา

Loop นี้ดูธรรมดา แต่เป็นเส้นแบ่งระหว่าง agent ที่ “พูดเหมือนทำ” กับ agent ที่ “ทำแล้วพิสูจน์” ในโรงเรียน Oracle ข้อหลังสำคัญกว่าเสมอ

ถ้า loop ขาดหนึ่งข้อ งานจะเริ่มเป่ Read หายจะกลายเป็น hallucination Plan หายจะกลายเป็นงานและ Patch หายจะมีแต่คำสัญญา Test หายจะไม่มี proof Report หาย Axe จะไม่รู้ว่าจะเชื่ออะไร

ดังนั้นทุก skill ที่ดีควรซ่อน loop นี้ไว้ข้างใน ผู้ใช้สั่งสั้นได้ แต่ agent ต้องทำครบ

## บทที่ 4 — AGENTS.md คือกฎของบ้าน

AGENTS.md คือไฟล์ที่บอก AI ว่าบ้านนี้ทำงานอย่างไร มันไม่ใช่ README สำหรับมนุษย์อย่างเดียว และไม่ใช่ที่เก็บความลับ มันคือ operating contract ของ repo

ในบ้าน Atom กฎสำคัญคือพูดไทย กระชับ สุภาพ ห้ามใช้คำหยาบ ห้ามเดาเรื่อง memory ต้องค้น ARRA ก่อน และต้อง verify ก่อน claim กฎพวกนี้ถ้าอยู่แค่ในแชท มันจะหายเมื่อ context หหมด แต่ถ้าอยู่ใน AGENTS.md agent รันทัดไปยังอ่านได้

AGENTS.md ที่ดีควรมีสิ่งที่ทำซ้ำทุก session เช่น command ที่ต้องรันหลังแก้ไฟล์ รูปแบบรายงาน ขอบเขต production กฎ secret และวิธีส่ง deliverable กลับ Discord

AGENTS.md ที่ไม่ดีคือไฟล์ที่ยาวจนไม่มีใครจับแทนได้ หรือใส่เรื่องใช้ครั้งเดียวจนกลบกฎสำคัญ หรือร้ายที่สุดคือใส่ token/password ลงไป กฎของบ้านต้องช่วยให้ปลอดภัย ไม่ใช่เพิ่มความเสี่ยง

ในงานหนังสือ AGENTS.md ทำหน้าที่เหมือนบรรณาธิการเงิบ มันบังคับให้ Atom ไม่ส่งแค่ path local เมื่อมี PDF บังคับให้ใช้ Sarabun/Fira Code สำหรับ book work และบังคับให้บอกว่า Atom เป็น AI Oracle ไม่ใช่มนุษย์

จุดที่ควรสอน Axe คือ AGENTS.md ไม่ได้ทำให้ AI ฉลาดขึ้นแบบเวทมนตร์ แต่มันลดความผิดพลาด เหมือน checklist cockpit นักบินเก่งก็ยังใช้ checklist เพราะสิ่งที่พลาดบ่อยไม่ควรฝากไว้กับอารมณ์หรือความจำ

ถ้า repo ไหนเริ่มมี agent หลายตัว AGENTS.md ยิ่งสำคัญ เพราะมันทำให้ทุกตัวมีรากเดียวกัน ห้องต่างกัน task ต่างกัน แต่ identity และ operating rule ต้องไม่แตก

สรุป: README บอกคนว่า project คืออะไร AGENTS.md บอก AI ว่าต้องทำตัวอย่างไรใน project นี้

## บทที่ 5 — Skill คือสูตร ไม่ใช่คำสั่งสวย ๆ

Skill คือ workflow ที่ใช้ซ้ำได้ ถ้างานหนึ่งมีขั้นตอนซ้ำ มี trap ซ้ำ และมี output contract ชัด งานนั้นควรถูกยกเป็น skill

ตัวอย่างที่ชัดคือ `/learn` มันไม่ได้แค่ clone repo แต่มันมี pattern ครบ: clone ด้วย ghq, symlink origin, สร้าง docs dir, spawn agents อ่านแยกมุม, เขียนไฟล์, สร้าง hub, และบันทึก lesson ที่สำคัญคือ skill เตือน bug ที่เคยเจอ: ถ้าส่งให้ agent แค่ `origin/` มันอาจเขียนไฟล์ผิด repo ดังนั้นต้องให้ทั้ง `SOURCE_DIR` และ `DOCS_DIR` แบบ absolute path

อีกตัวอย่างคือ `/dig` มันไม่ได้แค่เปิด session log แต่มันสอนว่า duration ดิบหลุดได้ ต้องดู gap, timeline, repo attribution และแยก human messages ออกจาก tool calls

Skill ที่ดีจึงมีทั้งขั้นตอนและภูมิคุ้มกัน มันไม่ใช่ prompt template ธรรมดา มันคือบทเรียนที่ถูกทำให้ executable ทางพฤติกรรม

เวลาสร้าง skill ให้ถามห้าข้อ

หนึ่ง งานนี้เกิดซ้ำไหม ถ้าเกิดครั้งเดียว อย่าเพิ่งทำ skill

สอง input คืออะไร URL, repo path, file, issue, หรือคำสั่งธรรมชาติ

สาม output ต้องเป็นอะไร Markdown, PDF, PR, report, zip, หรือ memory lesson

สี่ verify อย่างไร test, build, render, readback, search, หรือ visual check

ห้า trap ที่เคยเจอคืออะไร symlink, stale docs, local path, credential, production, context overflow

ในกรณีหนังสือ Skill ที่ถูกคือ `oracle-write-complete-book` ไม่ใช่ `oracle-cheatsheet` เพราะหนังสือต้องมี outline, draft, render, review และ PDF ส่วน cheatsheet เป็นภาคผนวกที่ช่วยจำหลังอ่านเล่ม

สรุปสั้น ๆ: prompt ทำให้ agent ทำครั้งนี้ skill ทำให้ agent ทำซ้ำได้โดยพลาดน้อยลง

## บทที่ 6 — Memory ที่เชื่อได้ต้อง Retrieve + Prove

---

AI ชอบพูดว่า “จำได้” แต่ในบ้าน Atom คำนี้ยังไม่พอ เราต้องมี memory loop: Record, Retrieve, Prove

Record คือบันทึกบทเรียนที่มีความหมายลง path ที่ถูก เช่น `ψ/memory/learnings/` หรือ runbook ไม่ใช่ปล่อยไว้ในแชท เพราะแชทเป็นบริบทชั่วคราว พอ compact หรือ rotate ก็หายจากการมองเห็นทันที

Retrieve คือก่อนตอบเรื่อง memory, prior work, identity, rule, lesson, decision หรือสิ่งที่เคยสอน ต้องค้น ARRA/Muninn ก่อน ถ้าไม่ค้นแล้วตอบจากความรู้สึก นั่นคือการเดา ไม่ใช่ memory

Prove คืออย่า claim ว่า index แล้วถ้ายังไม่มี readback ในงาน Orange Book รอบนี้ Atom บันทึก lesson แล้วมี ARRA id กลับมา นั่นคือหลักฐานว่าความรู้ถูกใส่เข้า memory layer จริง

Memory ที่ดีไม่ใช่การจำทุกอย่าง แต่คือการจำสิ่งที่กลับมาใช้ได้ เช่น token YouTube อยู่ที่ไหนต้องไม่พิมพ์ secret workflow OAuth ทำอย่างไร หรือ `/learn` repo นี้ได้ insight อะไร

สิ่งที่ไม่ควรจำคือ noise log ดิบยาว ๆ ที่ไม่มีบทเรียน secret ข้อมูล private ที่ไม่ควรข้ามห้อง หรือสรุปที่ยังไม่มี proof

ARRA จึงไม่ใช่กล่องเก็บ nostalgia แต่มันคือ intelligence loop ทำให้ Oracle รุ่นถัดไปไม่ต้องเรียนผิดซ้ำ และทำให้ Axe ถามย้อนหลังได้ว่า “ตอนนั้นสอนอะไรไว้” โดยมีหลักฐานมากกว่าคำตอบลอย ๆ

สำหรับหนังสือ Memory มีอีกบทบาทหนึ่ง มันทำให้เล่มไม่เป็นแค่บทความเดียว แต่เป็น artifact ที่เชื่อมกับประวัติงานจริง หนังสือที่ไม่มี memory อาจอ่านสิ้น แต่ตรวจต่อยาก หนังสือที่มี memory ก็จะกลายเป็นสะพานจากอดีตไปงานถัดไป



## บทที่ 7 — งานยาวต้องมี Goal และ Checkpoint

---

งานสั้นจบได้ในหนึ่ง turn แต่งานหนังสือ งานเว็บ งาน deploy หรือการแก้ระบบใหญ่ไม่ควรฝากไว้กับ context เดียว เพราะ context หกได้ session หมุนได้ และ agent ใหม่อาจไม่รู้ว่าทำถึงไหน

Goal ที่ดีต้องมี objective, success criteria, evidence และ stop condition ถ้าขาด objective งานจะไหล ถ้าขาด success criteria จะไม่รู้ว่าเสร็จเมื่อไหร่ ถ้าขาด evidence จะเหลือแค่คำกล่าวอ้าง ถ้าขาด stop condition agent อาจทำเกินเส้นปลอดภัย

ตัวอย่าง goal ที่ดีสำหรับหนังสือคือ: สร้างหนังสือ Codex Orange Book ฉบับ Atom สอน Axe เป็น PDF ที่อ่านได้จริง มี outline มี chapter source มี render log และมีไฟล์ได้ `/home/axezi/atom` เพื่อให้ Discord bridge แบนได้

Checkpoint คือจุดพักที่ทำให้คนถัดไปรับงานต่อได้ เช่น outline เสร็จ, draft เสร็จ, PDF render ผ่าน, review ผ่าน, ARRA indexed แต่ละ checkpoint ควรมีไฟล์หรือ command output ไม่ใช่ข้อความว่า “น่าจะเสร็จ”

งานยาวที่ไม่มี checkpoint มักพังแบบเงียบ ๆ ตอนแรกดูคืบหน้าเพราะมีข้อความเยาะ แต่พอถามหา artifact กลับไม่มีไฟล์ หรือพอลงมือทดสอบอะไรแล้วกลับตอบไม่ได้

ใน Codex/Oracle workflow checkpoint ยังช่วยป้องกัน context illusion agent อาจจำว่าได้ทำบางอย่างแล้ว แต่ filesystem ไม่โกหก ถ้าไฟล์ไม่มี แปลว่ายังไม่ได้ส่งมอบ

สรุปบทนี้: งานยาวต้องเขียนให้คนใหม่อ่านต่อได้ ถ้า agent ใหม่เปิดมาแล้วไม่รู้สถานะ แปลว่า checkpoint ยังไม่ดีพอ

# บทที่ 8 — Safety ไม่ใช่เบรกมือ แต่คือรางรถไฟ

หลายคนคิดว่า safety ทำให้งานช้า แต่สำหรับ agent ที่ทำงานบนเครื่องจริง safety คือสิ่งที่ทำให้งานไปได้ไกลโดยไม่พัง

เส้นแบ่งแรกคือ local reversible กับ external irreversible ถ้าเป็นไฟล์ local ที่แก้ย้อนกลับได้ อ่านได้ test ได้ ทำต่อได้ แต่ถ้าเป็น production deploy, payment, credential, account ภายนอก, force push, delete history หรือ secret ต้องหยุดที่ approval boundary

เส้นแบ่งที่สองคืออ่านกับเขียน การอ่านข้อมูลเพื่อสรุปมักปลอดภัยกว่า การเขียนออกไปยัง GitHub, Discord, YouTube, production service หรือ external API มี side effect ต้องพิสูจน์ก่อนว่าผู้ใช้สั่งจริงและสิทธิ์ถูกต้อง

เส้นแบ่งที่สามคือ local path กับ deliverable ใน Discord ผู้ใช้เปิด `/home/axezi/ ...` ไม่ได้ ดังนั้นงาน PDF, image, zip, markdown report ต้องเป็นไฟล์จริงใน path ที่ bridge แนบ หรือเป็น URL ที่เข้าถึงได้จริง การส่ง path อย่างเดียวถือว่าส่งไม่ครบ

Full access เป็นเครื่องมือ ไม่ใช่เป้าหมาย เปิด full access เพื่อแก้เร็วขึ้นแต่ไม่ verify คือความเสี่ยง ใช้ workspace/local command ให้พอก่อน escalate เฉพาะเมื่อจำเป็นและอธิบายผลกระทบได้

Secret เป็นอีกจุดที่ห้ามประมาท หนังสือ runbook หรือ memory ไม่ควรมี token, client secret, OAuth code หรือ password บันทึกได้ว่า token อยู่ path ไหนและ permission เป็นอะไร แต่ห้ามพิมพ์ค่าลับออกแชท

Safety ที่ดีไม่ใช่การปฏิเสธทุกอย่าง แต่คือการแยกงานที่ทำต่อได้จากงานที่ต้องหยุดถาม ถ้าเป็น clear local reversible task ปล่อยให้ลุย ถ้าแตะสิทธิ์เจ้าของหรือผลถาวร ให้หยุดที่ขอบเขต

นี่คือเหตุผลที่ Atom ทำงานแบบ YOLO ได้บางกรณี แต่ไม่ใช่ทุกกรณี YOLO หมายถึงไม่ถามในเรื่องปลอดภัยและย้อนกลับได้ ไม่ได้หมายถึงทำทุกอย่างโดยไม่สนผลกระทบ

## บทที่ 9 — Multi-agent ช่วยได้เมื่อแบ่งงาน ถูก

---

Multi-agent ไม่ได้แปลว่าดีกว่าเสมอ ถ้างานเล็ก การแตก agent จะเพิ่ม overhead แต่ถ้างานมีหลายมุมที่อ่านแยกกันได้ เช่น architecture, snippets, quick reference, testing, API surface การใช้หลาย agent จะเร็วและครอบคลุมกว่า

`/learn --deep` เป็นตัวอย่างที่ดี repo เดียวกันถูกอ่านจากห้ามุม แล้วแต่ละ agent เขียนไฟล์ของตัวเอง ผลลัพธ์ไม่ปะปน และ main agent กลับมาสรุป hub กับ key insights

กุญแจสำคัญคือ leader ต้องเป็นเจ้าของ integration subagent ทำ slice ของตัวเอง แต่ leader ต้องตรวจว่าไฟล์มีจริง เนื้อหาไม่ขัดกัน และข้อสรุปใหญ่ไม่เกินหลักฐาน

ห้ามใช้ subagent เป็นข้ออ้างในการไม่อ่านเอง main agent ยังต้องอ่าน skill instructions และต้องเข้าใจ output ก่อน claim โดยเฉพาะงานที่มี safety หรือ memory rule

Multi-agent ที่ดีควรมี task boundary ชัด: ใครอ่านอะไร เขียนไฟล์ไหน ห้ามแตะ path ไหน และ report อะไรกลับมา ถ้าไม่ชัด agent จะเขียนซ้ำกัน หรือเลวร้ายกว่านั้นคือเขียนทับกัน

ในการเขียนหนังสือเต็มเล่ม pattern ที่พิสูจน์แล้วคือหนึ่งบทต่อหนึ่ง writer agent แล้วมี reviewer แยก แต่ถ้าอยู่ใน Codex App outside tmux และไม่มี team runtime ก็ยังทำแบบ solo ได้โดยเลียนแบบ pipeline: outline ก่อน แยก chapter files แล้ว render/test

บทเรียนคือ multi-agent เป็นเครื่องมือของ throughput ไม่ใช่เครื่องมือของความจริง ความจริงยังต้องมาจากไฟล์ หลักฐาน และการ verify เหมือนเดิม

## บทที่ 10 — Playbook สำหรับ Axe

ถ้า Axe จะใช้ Codex/Atom ให้คุ้มค่าที่สุด ให้คิดเป็น playbook ไม่ใช่คิดเป็นคำสั่งเดียว

เริ่มจากบอกเป้าหมายและขอบเขต เช่น “เรียน repo นี้แบบ deep แล้วทำ docs” หรือ “ทำหนังสือใหม่จากสิ่งที่เรียนมา” ถ้ามีคำว่าใหม่ ต้องมี artifact ใหม่ ถ้ามีคำว่าหนังสือ ต้องมี PDF หรือ source book ไม่ใช่ cheatsheet อย่างเดียว

ถ้าเป็นการเรียน repo ให้ใช้ `/learn` ถ้าเป็นการขุดอดีตให้ใช้ `/dig` ถ้าเป็นการเขียนหนังสือให้ใช้ `oracle-write-complete-book` ถ้าเป็นปกให้ใช้ `oracle-book-cover` ถ้าเป็น cheatsheet ให้ใช้ `oracle-cheatsheet` เครื่องมือผิกระดับจะได้ผลลัพธ์ผิกระดับ

หลังสั่งงาน ให้ดู evidence มากกว่าดูคำพูด มีไฟล์ไหม path อยู่ได้ `/home/axezi/atom` ไหม render ผ่านไหม ARRA readback มีไหม ถ้าเป็น repo มี test/build ผ่านไหม ถ้าเป็น Discord attachment มี bridge แบบจริงไหม

ถ้า agent ตอบเร็วเกินไปในงานใหญ่ ให้สงสัยว่าอาจยังไม่ได้ทำ artifact งานใหญ่ที่ดีควรทิ้งร่องรอยไว้ใน filesystem: outline, chapters, build log, PDF, review note, memory lesson

สำหรับ Atom เอง บทเรียนจากเล่มนี้คืออย่าลดโทษตัวเอง เมื่อผู้ใช้บอกให้ทำหนังสือ ต้องใช้ pipeline หนังสือ ถ้าจะลด scope ต้องบอกว่าเป็น mini book หรือ draft ไม่ใช่แกล้งเรียก cheatsheet ว่าหนังสือ

สุดท้าย Codex ที่ดีไม่ใช่ Codex ที่ตอบยาวที่สุด แต่คือ Codex ที่เปลี่ยนงานยุ่งให้เป็น artifact ที่อ่านได้ ตรวจสอบได้ และใช้ต่อได้ นั่นคือเหตุผลที่บ้านนี้ใช้คำว่า Oracle ไม่ใช่ chatbot หนังสือเล่มนี้จบด้วย playbook สั้น ๆ:

อ่านหลักฐานก่อน วางโครงก่อน สร้างไฟล์จริง render/test จริง บันทึกบทเรียน แล้วค่อยรายงาน

ถ้าทำครบทุกข้อนี้ งานของ AI จะไม่เป็นแค่คำตอบ แต่มันจะกลายเป็นของที่ Axe ส่งต่อได้

# ภาคผนวก — สูตรโง่งการทำ Oracle Book

---

# Oracle Book 3-Skill Pipeline สูตรโกง

ใช้ทุกครั้งเมื่อ Axe บอกว่า “ทำหนังสือ” หรือ “เขียนหนังสือ”

## ลำดับหลัก

```
oracle-write-complete-book  
→ oracle-book-cover  
→ oracle-cheatsheet  
→ final PDF attach
```

## oracle-write-complete-book

ใช้ทำตัวเล่มจริง:

- 1) ขุดหลักฐาน
- 2) ทำ OUTLINE.md
- 3) เขียน chapters/\*.md
- 4) รวม BOOK.md
- 5) render PDF
- 6) review

## oracle-book-cover

ใช้ทำปกและ visual:

- 1) IP/license gate
- 2) cover concept
- 3) render cover preview
- 4) bake cover เข้าเล่ม
- 5) export social crops 1:1 และ 4:5

## oracle-cheatsheet

ใช้ทำภาคผนวกท้ายเล่ม:

- command reference
- workflow ลัด
- traps ที่เจอจริง
- checklist สำหรับรอบถัดไป

### trap ที่ต้องจำ

อย่าใช้ cheatsheet แทนหนังสือ  
อย่าส่ง PDF ที่ยังไม่มี cover/layout pass  
อย่าบอกว่าจำแล้ว ถ้ายังไม่ ARRA readback

# Sources และข้อจำกัด

---

- Learned docs: `ψ/learn/alchaincyf/codex-orange-book/2026-06-30/1836_*.md`
- Official manual cache: `/tmp/openai-docs-cache/codex-manual.md`
- Session dig output: `/tmp/atom_dig_default.json`
- v1 draft: `ψ/writing/books/2026-06-30_codex-orange-book-atom/`

ข้อมูลผลิตภัณฑ์ Codex/OpenAI เปลี่ยนได้ ต้องตรวจ official docs ปัจจุบันก่อนใช้งานจริง เรื่อง model, pricing, quota และ feature availability.